

Design Pattern Report: Observer Pattern

Group 52

Nguyễn Đình Minh Huy (Student ID: 25125083)

July 20, 2026

Contents

1	Real-world Problem	2
2	Naive Solution (Without Design Pattern)	2
3	Problems with the Naive Solution	3
4	Introduction to the Observer Pattern	3
5	General Class Diagram	3
6	Class Diagram for the Bank Account Problem	4
7	Implementation of the Observer Pattern	5
8	Pros and Cons	6
8.1	Pros	6
8.2	Cons	6
9	Other Real-world Applications	7
9.1	Web Development	7
9.2	Mobile Development	7
10	Observer and Mediator Fusion	7
11	Quiz	8
12	References	9

1 Real-world Problem

Consider a modern financial core banking system where transactions are performed on a `BankAccount`. When a deposit occurs, several peripheral systems need to be notified of the event immediately to execute their processes:

- **UI:** Updates the account balance display on the user's screen.
- **Email System:** Sends a transaction alert email to the customer.
- **Logger:** Writes transaction details to the security audit trail.
- **Fraud AI:** Scans transaction metadata to detect suspicious patterns.
- **Mobile App:** Delivers a push notification to the client's phone.
- **Analytics:** Records transaction metrics for business intelligence.

A customer or administrator must be able to register, enable, or disable these notification channels dynamically. The core challenge is establishing a system where a bank account can broadcast transaction updates to an arbitrary and dynamic set of observer systems without knowing their concrete classes or internal notification mechanisms.

2 Naive Solution (Without Design Pattern)

In a naive implementation, the `BankAccount` class holds references to concrete notification system objects directly. Below is the C++ representation of this approach:

```
1 // Refer to Observer/src/naive.cpp for full implementation details
2 class NaiveBankAccount {
3 private:
4     double balance;
5     UI ui;
6     EmailSystem email;
7     Logger logger;
8     FraudDetector fraudDetector;
9     MobileApp mobileApp;
10    Analytics analytics;
11
12 public:
13     NaiveBankAccount(double initialBalance) : balance(initialBalance)
14     {}
15
16     void deposit(double amount) {
17         balance += amount;
18
19         // Direct, coupled method invocations on concrete systems
20         ui.update(balance);
21         email.sendEmail(amount);
22         logger.log(amount, balance);
23         fraudDetector.scan(amount);
24         mobileApp.pushNotification(amount);
25         analytics.trackActivity("Deposit", amount);
26     }
27 };
```

Listing 1: Naive Implementation of Bank Account Notifications

3 Problems with the Naive Solution

The naive approach exhibits several major architectural flaws:

1. **Tight Coupling:** The `NaiveBankAccount` is tightly coupled to concrete classes like `UI`, `EmailSystem`, `Logger`, `FraudDetector`, `MobileApp`, and `Analytics`. It must know their names, structures, and exactly which update method to call on each.
2. **Violation of the Open-Closed Principle (OCP):** If we want to introduce a new system (such as a compliance reporting system or a credit score monitoring engine), we must modify the core `NaiveBankAccount` class. This requires adding a new member field and updating the notification logic inside the `deposit` method.
3. **Violation of the Single Responsibility Principle (SRP):** The `BankAccount` class should focus solely on financial ledger operations (managing the balance and transactions). However, it is also burdened with orchestrating emails, logs, mobile push notifications, and security analysis.

4 Introduction to the Observer Pattern

The **Observer Pattern** is a behavioral design pattern that defines a one-to-many dependency between objects. When one object (the **Subject** or **Publisher**) changes state, all its dependents (the **Observers** or **Subscribers**) are notified and updated automatically.

Key concepts:

- **Subject:** Knows its observers and provides an interface to attach or detach them.
- **Observer:** Defines an updating interface for objects that should be notified.
- **ConcreteSubject:** Stores state of interest to `ConcreteObservers` and sends notifications.
- **ConcreteObserver:** Implements the `Observer` updating interface to keep its state consistent with the subject.

5 General Class Diagram

In the general `Observer` pattern structure:

- A `Subject` maintains a list of references to `Observer` objects.
- The `Subject` exposes interfaces: `attach(Observer)`, `detach(Observer)`, and `notify()`.
- `Observer` exposes the `update()` method interface.

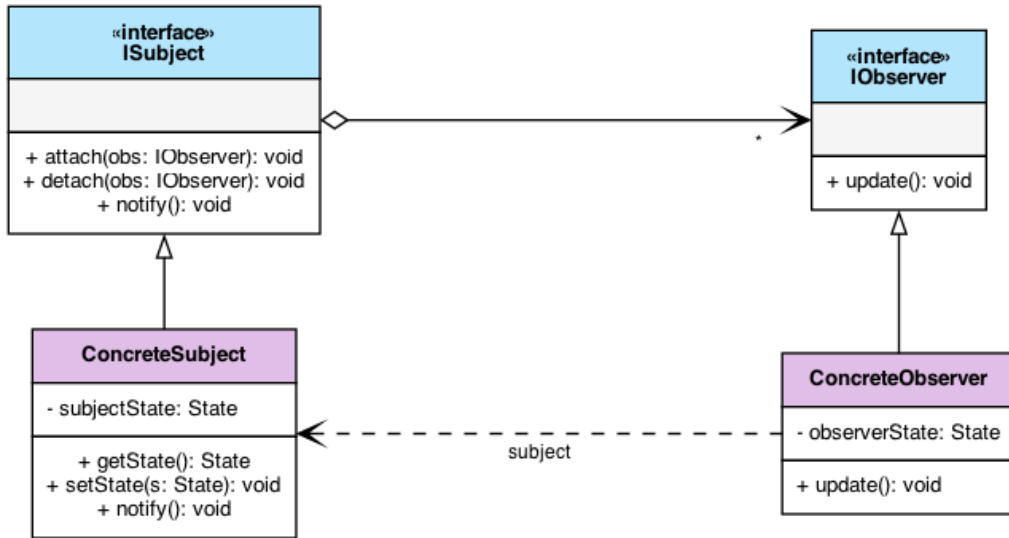


Figure 1: General Observer Design Pattern UML Diagram

6 Class Diagram for the Bank Account Problem

Applying the pattern to our problem:

- **BankAccount** acts as the **ConcreteSubject** (or subject coordinator).
- **AccountObserver** acts as the **Observer interface**.
- **UIObserver**, **EmailObserver**, **LoggerObserver**, **FraudAIObserver**, **MobileAppObserver**, and **AnalyticsObserver** act as the **ConcreteObservers**.

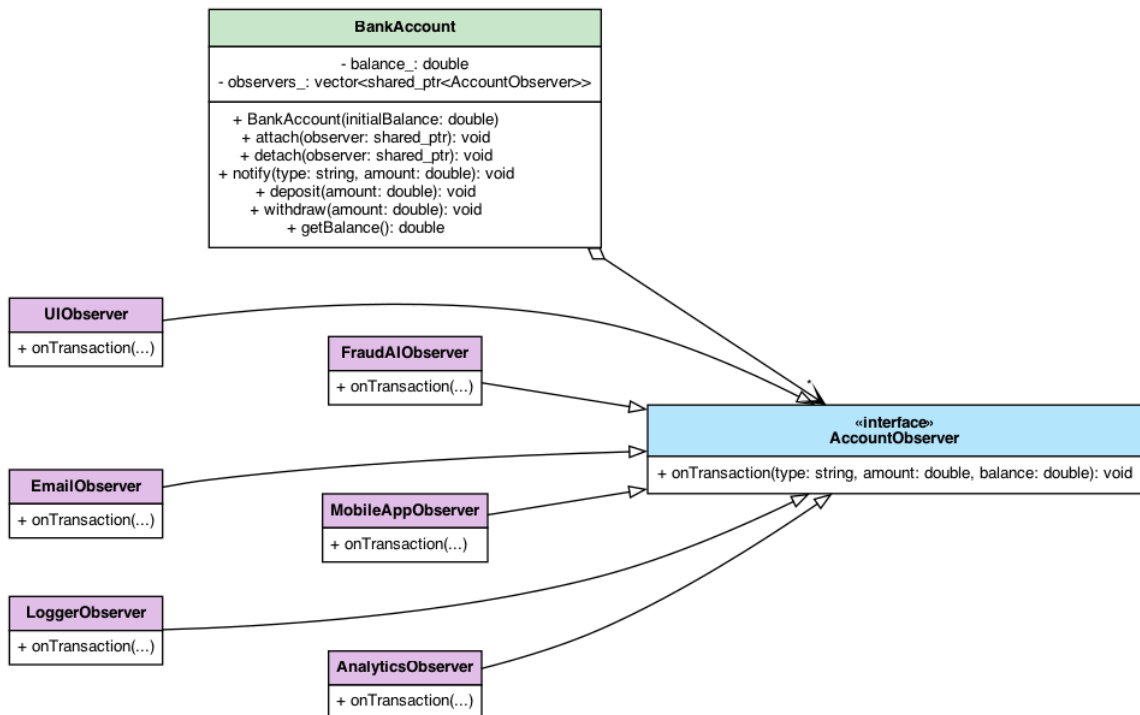


Figure 2: UML Class Diagram for Bank Account Notification System

7 Implementation of the Observer Pattern

By decoupling the classes through abstract interfaces, our refactored implementation allows adding or removing subscriber systems at runtime without modifying the core subject.

```
1 // Refer to Observer/src/pattern.cpp for full implementation details
2 class AccountObserver {
3 public:
4     virtual ~AccountObserver() = default;
5     virtual void onTransaction(const std::string& type, double amount,
6         double currentBalance) = 0;
7 };
8 class BankAccount {
9 private:
10     double balance;
11     std::vector<std::shared_ptr<AccountObserver>> observers;
12
13 public:
14     BankAccount(double initialBalance) : balance(initialBalance) {}
15
16     void attach(std::shared_ptr<AccountObserver> obs) { observers.
17         push_back(obs); }
18     void detach(std::shared_ptr<AccountObserver> obs) { /* Remove from
19         list */ }
20     void notifyObservers(const std::string& type, double amount) {
21         for (const auto& obs : observers) {
22             obs->onTransaction(type, amount, balance);
23         }
24     }
25
26     void deposit(double amount) {
27         balance += amount;
28         notifyObservers("Deposit", amount);
29     }
30 };
```

Listing 2: Refactored Observer Implementation

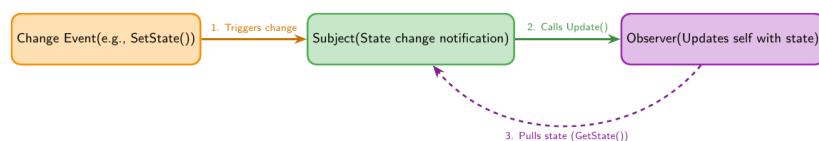


Figure 3: Sequential Process Flowchart of a Single Observer Notification

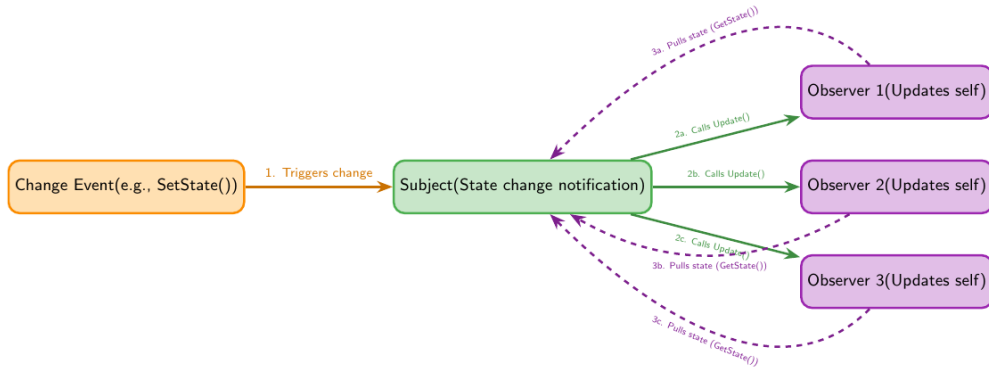


Figure 4: Sequential Process Flowchart with Multiple Observers (Pull Model)

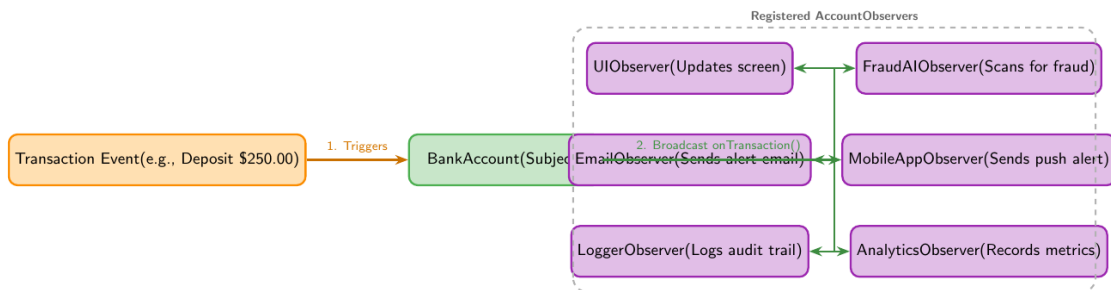


Figure 5: Process Flowchart for Bank Account Notification System (Push Model)

8 Pros and Cons

8.1 Pros

- **Loose Coupling:** The Subject and Observers are loosely coupled. The Subject only knows that the observer implements a specific interface.
- **Support for Broadcast Communication:** Unlike direct calls, notifications are broadcast automatically to all registered subscribers.
- **Open-Closed Principle:** We can introduce new concrete observer classes without changing the subject's code.

8.2 Cons

- **Lapsed Listener Problem:** Observers must be carefully deregistered when destroyed, or they remain in memory and get called, causing undefined behavior or leaks.
- **Unintended Update Chains:** If observers trigger updates on subjects, it can lead to complex and hard-to-debug cascading updates or infinite loops.

- **Ordering Concerns:** There is no guarantee in which order the observers will be notified.

9 Other Real-world Applications

9.1 Web Development

- **Event Handling in JavaScript:** The DOM's `addEventListener` uses the Observer pattern to attach event handlers to HTML elements.
- **State Management (Redux/Vuex):** Views subscribe to changes in a global store and re-render automatically when state mutations occur.

9.2 Mobile Development

- **RxSwift / RxJava / RxKotlin:** Reactive streams rely heavily on observing data sequences and binding them directly to UI components.
- **SwiftUI/Android Jetpack Compose:** Declarative UI frameworks use property wrappers like `@Published` and state flows to push model updates directly to the view.

10 Observer and Mediator Fusion

In complex system architectures, the *Observer* and *Mediator* patterns are frequently fused together to handle coordination and communication between multiple components.

In a standard Mediator implementation, colleagues must explicitly call the mediator, and the mediator directly invokes actions on other colleagues. By fusing the two patterns:

- **Colleagues act as Subjects:** They maintain state and notify registered observers when changes occur, remaining completely unaware of who is listening.
- **The Mediator acts as an Observer:** It subscribes to the event streams of the Colleagues. When it receives an `update()` notification, it coordinates and executes the required reaction across other Colleagues.

This fusion achieves dynamic event-driven routing while preserving loose coupling between Colleagues.

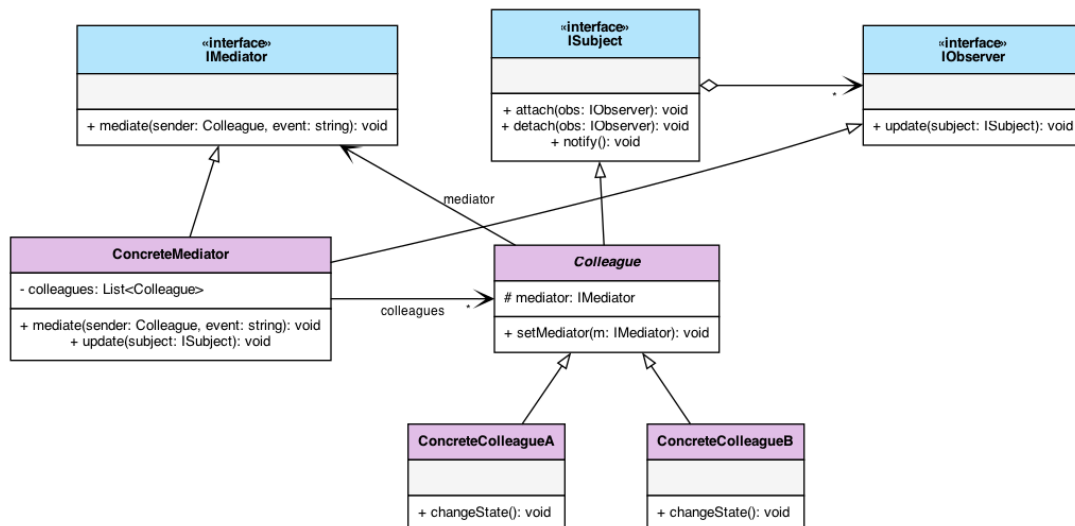


Figure 6: UML Class Diagram of the Observer-Mediator Fusion

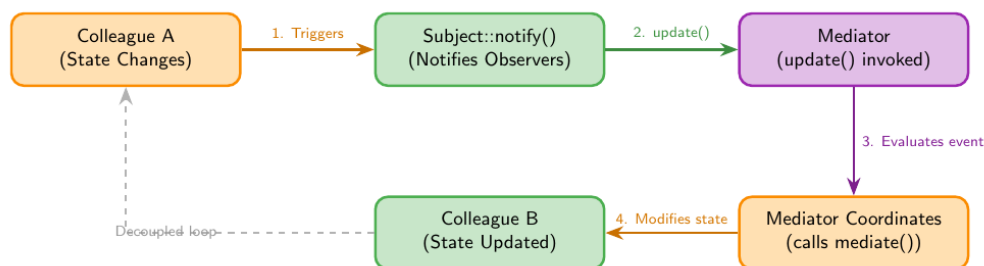


Figure 7: Process Flowchart of Event-Driven Mediation Flow

11 Quiz

Here are some interactive questions to review the Observer pattern:

1. What type of design pattern is the Observer Pattern?
 - A) Creational
 - B) Structural
 - C) Behavioral
 - D) Architectural
2. Which SOLID principle does the Observer pattern primarily help satisfy when adding new subscribers?
 - A) Liskov Substitution Principle
 - B) Single Responsibility Principle
 - C) Interface Segregation Principle
 - D) Open-Closed Principle

3. What is the “Lapsed Listener” problem in the Observer pattern?
- A) The subject stops listening to updates.
 - B) Observers are not notified because the subject crashed.
 - C) An observer is deleted without unsubscribing, causing memory leaks/crashes.
 - D) Multiple subjects occupy the same listener list.
4. Which of the following is a key advantage of the Observer pattern?
- A) It ensures observers are updated sequentially.
 - B) It decouples the subject from concrete observer details.
 - C) It reduces memory overhead of notifications.
 - D) It prevents subjects from changing their state.

Answer Key: 1-C, 2-D, 3-C, 4-B.

12 References

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Refactoring.Guru. *Design Patterns: Observer*. <https://refactoring.guru/design-patterns/observer>